Model Selection and Architecture

EduGenie - AI Learning Assistant

1. Objective

This stage decided which AI models would power EduGenie and how the application would be structured around them. The aim was a design that is cheap to run, easy to understand, robust against failures and simple to extend with new learning tools.

2. Design Requirements

- Offer five different study aids (explain, answer, summarise, quiz, roadmap) from one interface.
- Keep at least one feature working without paying per request.
- Return readable, well-structured answers suited to students.
- Let the user see which model produced each answer.
- Fail gracefully when a model, the network or the API key has a problem.

3. Model Selection and Justification

Aspect	LaMini-Flan-T5-783M (local)	Gemini Flash (cloud)
Provider / access	MBZUAI, downloaded from Hugging Face	Google, called through the google-genai client
Runs on	The user's own CPU	Google Cloud servers
Cost	No per-request cost	Based on API usage and quota
Strength	Small, fast to start, works offline after download	Follows long, structured instructions; fluent long-form output
Weakness	Short, simpler answers	Needs internet and an API key
Used for	Concept Explanation	Q&A, Summarisation, Quiz, Learning Path

How the choice was made. Concept explanation needs only a few clear sentences, which an instruction-tuned T5 model of this size can produce locally. The other four tasks require strict output layouts - for example a quiz with options A-D and an Answer Key, or a roadmap with phases and timelines - and longer answers. A larger cloud model follows such formatting instructions far more reliably, so those tasks use Gemini.

Why it is useful. This hybrid approach gives students a free tool for quick definitions and a stronger tool for deeper study material, while the interface makes the difference visible through the model badge.

4. System Architecture

Presentation layer	templates/index.html and static/style.css. A form for input and an answer box for output.

API layer	Main.py with FastAPI: GET / and POST /generate, the TaskType enum, start-up checks and error handling.
Feature layer	explanation_module, qna, summary_module, quiz_module, learning_path - one file per learning tool.
Model layer	Local: Transformers tokenizer and AutoModelForSeq2SeqLM. Cloud: one shared Gemini client and the generate_with_retry helper.
Configuration	python-dotenv loads the key; Main.py validates it before the server accepts requests.

5. Module Responsibilities

File	Responsibility
Main.py	Creates the app, mounts static files and templates, validates the API key, defines the TaskType enum, routes requests and renders results.
explanation_module.py	Loads the local model once (lazy loading with module-level cache) and generates concept explanations.
qna.py	Wraps the user's question in a prompt asking for clear, detailed answers with examples.
summary_module.py	Builds a prompt that requests an overview first, then bullet-point takeaways in simple language.
quiz_module.py	Builds a prompt for a 3-question multiple-choice quiz with an Answer Key.
learning_path.py	Builds a prompt for a phased roadmap with topics, project ideas and timelines in Markdown.
gemini_helper.py	Provides generate_with_retry, shared by all Gemini modules (file not included in the code supplied for this document, so only its use is described).

6. Key Architectural Decisions

Decision	Reason	Benefit
One module per task	Each tool has its own prompt and rules	A change to the quiz cannot break the summary
Gemini client created once and passed in as a parameter	Avoids repeated set-up and global state	Modules are easy to test with a fake client
Lazy loading of the local model	Loading takes time and memory	Fast start-up; model loaded only when needed
Module-level cache (_tokenizer, _model)	Avoid reloading on each request	Only the first concept request is slow
Shared retry helper	Cloud calls can fail temporarily	One place handles retries for four features
Fail-fast key validation	A bad key would otherwise fail mid-request	Clear message at start-up
Model label sent with every answer	Transparency for the learner	The page shows which model answered

7. Outcome

The result is a layered, modular architecture in which a thin FastAPI layer routes each request to a dedicated feature module, and each module talks to the model best suited to the job. This structure is what the next stage, Core Functionalities Development, builds on.